차세대 반도체 디지털 혁신공유대학 사업단
POLARIS

2026년 Deep Learning Hardware 설계 경진대회

2026.02.02 (Monday)

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

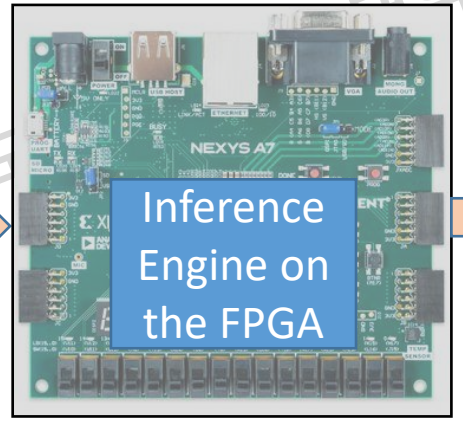
Outlines

- Recap
- System Overview
- Test Vector Generation
- Sample Top Design

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

설계의 목표

무인판매대에서 상품 인식을 위하는 딥러닝 추론 하드웨어를 설계한다.



Copyright (차세대반도체 혁신공유대학 사업단) all rights reserved.

Software Development Kit (SDK)

- Software for model quantization
  - C/C++: OS: Windows (Visual Studio (2019[1])), Linux (gcc (7.5.0[2]) on Ubuntu 18.04)), Mac (gcc)
  - Python (3.7.9[3] or 2.7.17[4])
    - Automatically generate the directories for test images.
    - Optional: Analyze data for quantization.
  - *(1),(2),(3),(4)* ***the code is likely to work fine with other versions. We will give additional support if needed.***
- Hardware implementation
  - Xilinx Vivado 2021.1:
    - RTL simulation and verification, IP Packaging, Synthesis, and FPGA implementation.
    - Only support Windows and Linux versions. *For a Mac OS, need to install a virtual machine*

  ⇒ *You should utilize a desktop PC in the laboratories at your school*
- System Integration
  - Host code: (1) C/C++, or (2) Python
  - Xilinx Vitis 2021.1 (C/C++)
    - Make the firmware code

Copyright 2022-2026. (사)세대반도체 혁신공유대학 사업단, all rights reserved.

Software Code Structure: Windows/Unix

skeleton

- **3rdparty : Third party library (Don't touch)**
  - lib/ Library
  - include/ Header files
  - dll/ Dynamic Link Library
  - CLBlast/ A modern, lightweight, performant and tunable OpenCL BLAS library implements BLAS routines: *basic linear algebra subprograms* (BLAS) operating on vectors and matrices.
- **src : Source code**
  - main.c //Main file
  - yolov2_forward_network.c // Do inference for an **FP32** model
  - yolov2_forward_network_quantized.c // Do inference for an **int8 quantized** model
  - additionally.c // All functions and utilities
  - …
- **bin : Executable files and bash scripts**
  - dataset/ test images and labels, make_list_cur.py, show_images.py, size_search.py, target.txt
  - weights/ output files when saving the model's parameters layer by layer
  - *.weights, *.cfg Files to save the parameters, and the structure of the model AIX2023
  - … Scripts for Unix/Linux (*.sh), scripts for Windows (*.cmd), executable files
- **obj : Object files when compiled on Unix/Linux (Don't care)**
- **Makefile : Makefile**
- ***.sln, *.cv* : Visual studio project**

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Compile and Run: Windows

- Run a script to **test the AIX2024 model** **on one image**

script-wins-aix2024-test-one.cmd

yolo_cpu.exe detector test yolohw.names aix2024.cfg aix2024.weights -thresh 0.24 test01.jpg -out_filename test01-det

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Compile and Run: Windows

- Run a script to **test the AIX2024 model** **on all test images**

script-wins-aix2024-test-all.cmd

yolo_cpu.exe detector map yolohw.names aix2024.cfg aix2024.weights -thresh 0.24

```
C:₩skeleton₩bin>script-wins-aix2024-test-all.cmd

C:₩skeleton₩bin>yolo_cpu.exe detector map yolohw.names aix2024.cfg aix2024.weights -thresh 0.24
valid: Using default 'C:/skeleton/bin/dataset/target.txt'
names: Using default 'C:/skeleton/bin/yolohw.names'
layer     filters    size              input                output
    0 conv     16  3 x 3 / 1   256 x 256 x   3   ->   256 x 256 x  16 0.057 BF
    1 max          2 x 2 / 2   256 x 256 x  16   ->   128 x 128 x  16
    2 conv     32  3 x 3 / 1   128 x 128 x  16   ->   128 x 128 x  32 0.151 BF
    3 max          2 x 2 / 2   128 x 128 x  32   ->    64 x  64 x  32
    4 conv     64  3 x 3 / 1    64 x  64 x  32   ->    64 x  64 x  64 0.151 BF
    5 max          2 x 2 / 2    64 x  64 x  64   ->    32 x  32 x  64
    6 conv    128  3 x 3 / 1    32 x  32 x  64   ->    32 x  32 x 128 0.151 BF
    7 max          2 x 2 / 2    32 x  32 x 128   ->    16 x  16 x 128
    8 conv    256  3 x 3 / 1    16 x  16 x 128   ->    16 x  16 x 256 0.151 BF
    9 max          2 x 2 / 2    16 x  16 x 256   ->     8 x   8 x 256
   10 conv    512  3 x 3 / 1     8 x   8 x 256   ->     8 x   8 x 512 0.151 BF
   11 max          2 x 2 / 1     8 x   8 x 512   ->     8 x   8 x 512
   12 conv    256  1 x 1 / 1     8 x   8 x 512   ->     8 x   8 x 256 0.017 BF
   13 conv    512  3 x 3 / 1     8 x   8 x 256   ->     8 x   8 x 512 0.151 BF
   14 conv    195  1 x 1 / 1     8 x   8 x 512   ->     8 x   8 x 195 0.013 BF
   15 yolo
   16 route  12
   17 conv    128  1 x 1 / 1     8 x   8 x 256   ->     8 x   8 x 128 0.004 BF
   18 upsample            2x     8 x   8 x 128   ->    16 x  16 x 128
   19 route  18 8
   20 conv    195  1 x 1 / 1    16 x  16 x 384   ->    16 x  16 x 195 0.038 BF
   21 yolo
Total BFLOPS 1.035
Loading weights from aix2024.weights...
Done!
```

```
class_id = 43, name = campbells_chicken_noodle_soup,      ap = 99.47 %
class_id = 44, name = frappuccino_coffee,         ap = 91.53 %
class_id = 45, name = chewy_dips_chocolate_chip,          ap = 64.94 %
class_id = 46, name = chewy_dips_peanut_butter,           ap = 89.97 %
class_id = 47, name = nature_vally_fruit_and_nut,         ap = 92.30 %
class_id = 48, name = cheerios,           ap = 96.27 %
class_id = 49, name = lindt_excellence_cocoa_dark_chocolate,      ap = 81.82 %
class_id = 50, name = hersheys_symphony,          ap = 100.00 %
class_id = 51, name = campbells_chunky_classic_chicken_noodle,    ap = 94.65 %
class_id = 52, name = martinellis_apple_juice,    ap = 79.72 %
class_id = 53, name = dove_pink,          ap = 74.48 %
class_id = 54, name = dove_white,         ap = 88.93 %
class_id = 55, name = david_sunflower_seeds,      ap = 95.94 %
class_id = 56, name = monster_energy,     ap = 44.72 %
class_id = 57, name = act_ii_butter_lovers_popcorn,       ap = 86.10 %
class_id = 58, name = coca_cola_glass_bottle,     ap = 81.61 %
class_id = 59, name = twix,       ap = 85.90 %
 for thresh = 0.24, precision = 0.74, recall = 0.64, F1-score = 0.69
 for thresh = 0.24, TP = 1895, FP = 671, FN = 1064, average IoU = 55.01 %

 mean average precision (mAP) = 0.817559, or 81.76 %
Total Detection Time: 7.000000 Seconds
```

mAP = 81.76%

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Test quantization: Windows

- Run a script to **test the AIX2024 model** **on all test images**

script-wins-aix2024-test-all-quantized.cmd

yolo_cpu.exe detector map yolohw.names aix2024.cfg aix2024.weights -thresh 0.24 -quantized

Full-Precision

```
class_id = 43, name = campbells_chicken_noodle_soup,      ap = 99.47 %
class_id = 44, name = frappuccino_coffee,         ap = 91.53 %
class_id = 45, name = chewy_dips_chocolate_chip,          ap = 64.94
class_id = 46, name = chewy_dips_peanut_butter,           ap = 89.97 %
class_id = 47, name = nature_vally_fruit_and_nut,         ap = 92.30 %
class_id = 48, name = cheerios,           ap = 96.27 %
class_id = 49, name = lindt_excellence_cocoa_dark_chocolate,      ap = 81.82 %
class_id = 50, name = hersheys_symphony,          ap = 100.00 %
class_id = 51, name = campbells_chunky_classic_chicken_noodle,    ap = 94.65 %
class_id = 52, name = martinellis_apple_juice,    ap = 79.72 %
class_id = 53, name = dove_pink,          ap = 74.48 %
class_id = 54, name = dove_white,         ap = 88.93 %
class_id = 55, name = david_sunflower_seeds,      ap = 95.94 %
class_id = 56, name = monster_energy,     ap = 44.72 %
class_id = 57, name = act_ii_butter_lovers_popcorn,       ap = 86.10 %
class_id = 58, name = coca_cola_glass_bottle,     ap = 81.61 %
class_id = 59, name = twix,       ap = 85.90 %
 for thresh = 0.24, precision = 0.74, recall = 0.64, F1-score = 0.69
 for thresh = 0.24, TP = 1895, FP = 671, FN = 1064, average IoU = 55.01 %

 mean average precision (mAP) = 0.817559, or 81.76 %
Total Detection Time: 7.000000 Seconds
```

mAP = 81.76%

After Quantization (INT8)

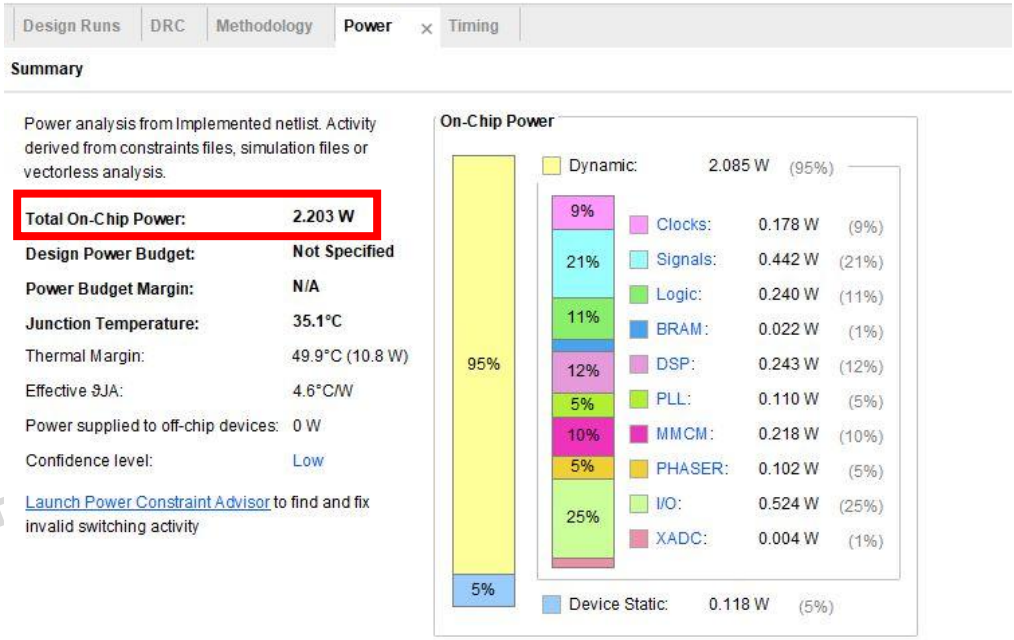
```
class_id = 43, name = campbells_chicken_noodle_soup,      ap = 45.40 %
class_id = 44, name = frappuccino_coffee,         ap = 80.00 %
class_id = 45, name = chewy_dips_chocolate_chip,          ap = 35.23 %
class_id = 46, name = chewy_dips_peanut_butter,           ap = 72.98 %
class_id = 47, name = nature_vally_fruit_and_nut,         ap = 24.98 %
class_id = 48, name = cheerios,           ap = 87.83 %
class_id = 49, name = lindt_excellence_cocoa_dark_chocolate,      ap = 59.94 %
class_id = 50, name = hersheys_symphony,          ap = 98.99 %
class_id = 51, name = campbells_chunky_classic_chicken_noodle,    ap = 66.98 %
class_id = 52, name = martinellis_apple_juice,    ap = 32.10 %
class_id = 53, name = dove_pink,          ap = 22.91 %
class_id = 54, name = dove_white,         ap = 58.06 %
class_id = 55, name = david_sunflower_seeds,      ap = 71.00 %
class_id = 56, name = monster_energy,     ap = 10.08 %
class_id = 57, name = act_ii_butter_lovers_popcorn,       ap = 47.84 %
class_id = 58, name = coca_cola_glass_bottle,     ap = 65.94 %
class_id = 59, name = twix,       ap = 37.09 %
 for thresh = 0.24, precision = 0.64, recall = 0.23, F1-score = 0.34
 for thresh = 0.24, TP = 672, FP = 380, FN = 2287, average IoU = 44.83 %

 mean average precision (mAP) = 0.550382, or 55.04 %
Total Detection Time: 10.000000 Seconds
```

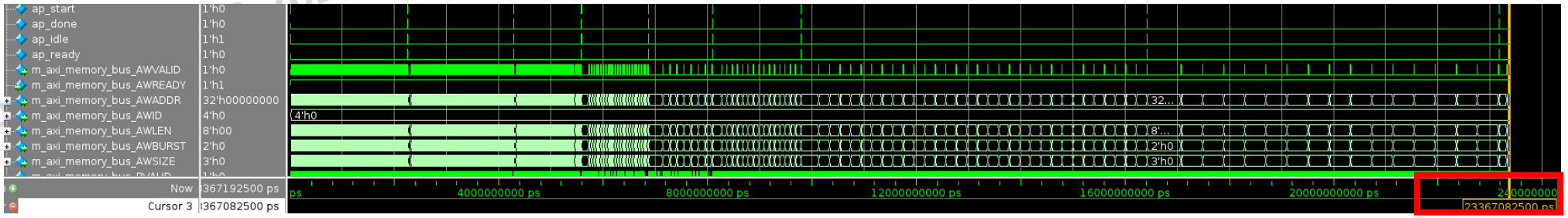
mAP = 55.04%

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

결과평가: Accuracy/Speed/Power

- **Accuracy:** Mean average precision (mAP)
  - Calculate the AP at IoU threshold 0.5 for INT8
- **Inference speed:** measures the number of frames your CNN accelerator IP processes in a second
  - RTL simulation result: 23.3@200MHz, 2 cycles for DRAM latency (Use the FPGA clock).
- **Energy:**
  - Implement your design on an FPGA board
  - Measure the on-chip power of the design after FPGA implementation
- Report the setting (Expected: frequency = 100MHz)





Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

결과평가: Accuracy/Speed/Power

- Objective evaluation[1]: **10^4 / Power * ReLU(mMAP – 0.2) * ReLU (fps – 5).**
  - Accuracy: Why “mMAP - 0.2”? An offset of 0.2 is added to encourage students to improve the accuracy of a quantized model.
    - **Baseline** Quantized model: 0.5504 - 0.2 = 0.3504 → You still got a good number to design HW without doing any quantization.
    - **Full-precision** model: 0.8176 - 0.2 = 0.6176 → which is about 1.75x better than the baseline quantized model.
    - If a quantized model achieves> 73.58%, it becomes 0.7358 - 0.2 = 0.5358 (about 86.75% of the full-precision model).

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

결과평가: Accuracy/Speed/Power

- Objective evaluation[(1)]: **10^4 / Power * ReLU(mMAP – 0.2) * ReLU (fps – 5).**
  - Performance: **Why "fps - 5"?** As a pretrained model has 1.035 billion operations, you are expected to design an engine with a reasonable number of MACs.
    - For example, use 36 MACs = (3x3) x (4), (e.g., computing four output pixels in parallel, with 9 MACs per output pixel), at 100MHz
      - → Peak throughput = 36 * 2 * 100 MHz = 7.2 GOPS
      - → Peak FPS = 7.2/1.035 = 6.96 > 5 (fps)
    - *If you choose a small number of MACs, the design can not achieve 5 fps@100MHz.*
- Important notes:
  - The fps is measured with an FPGA board or RTL simulation at 100 MHz.
    - For example, the latency is 46.6 ms at 100 MHz → the inference speed: 21.46 (= 1000/46.6) (fps)
  - Clearly report the frequency you achieved in advance.

Copyright 2022-2026. (차세대반도체혁신공유대학 사업단) all rights reserved.

결과평가: Accuracy/Speed/Power

- Objective evaluation(1): **10^4 / Power * ReLU(mMAP – 0.2) * ReLU (fps – 5).**
  - Power: It is included to confirm the FPGA implementation.
    - For example, the number 2.203 W is obtained with a design with 576 MACs.
    - However, reducing the MACs will not significantly change the overall power.
- Important notes:
  - Clearly report the frequency you achieved in advance.

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

Outlines

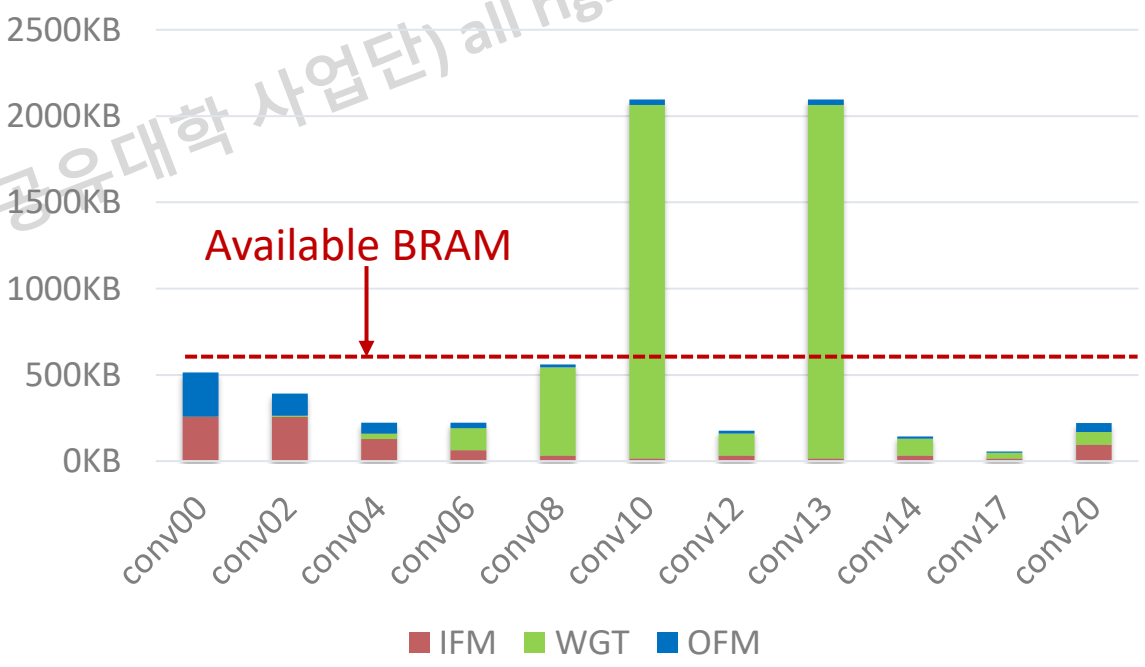
- Recap
- System Overview
- Test Vector Generation
- Sample Top Design

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

System Overview: Model Selection

- How do we design a model?
  - (1) Number of operations: 1.035 GOPs ➔ Achieve 10~25 inferences per second with 144 MACs
  - (2) The **first three layers** are small ➔ Store weights in ROM ➔ For the mid-term evaluation
  - (3) With a good design, activations can be stored in on-chip memory

Layer	Input	Output	Weight	Remark
0	256×256×3	256×256×16	3×3×3×16	ROM
2	128×128×16	128×128×32	3×3×16×32	ROM
4	64×64×32	64×64×64	3×3×32×64	ROM
6	32×32×64	32×32×128	3×3×64×128	DRAM/MIG
8	16×16×128	16×16×256	3×3×128×256	DRAM/MIG
10	8×8×256	8×8×512	3×3×256×512	DRAM/MIG
12	8×8×512	8×8×256	1×1×512×256	DRAM/MIG
13	8×8×256	8×8×512	3×3×256×512	DRAM/MIG
14	8×8×512	8×8×195	1×1×512×195	DRAM/MIG
17	8×8×256	8×8×128	1×1×256×128	DRAM/MIG
20	16×16×384	16×16×195	1×1×384×195	DRAM/MIG



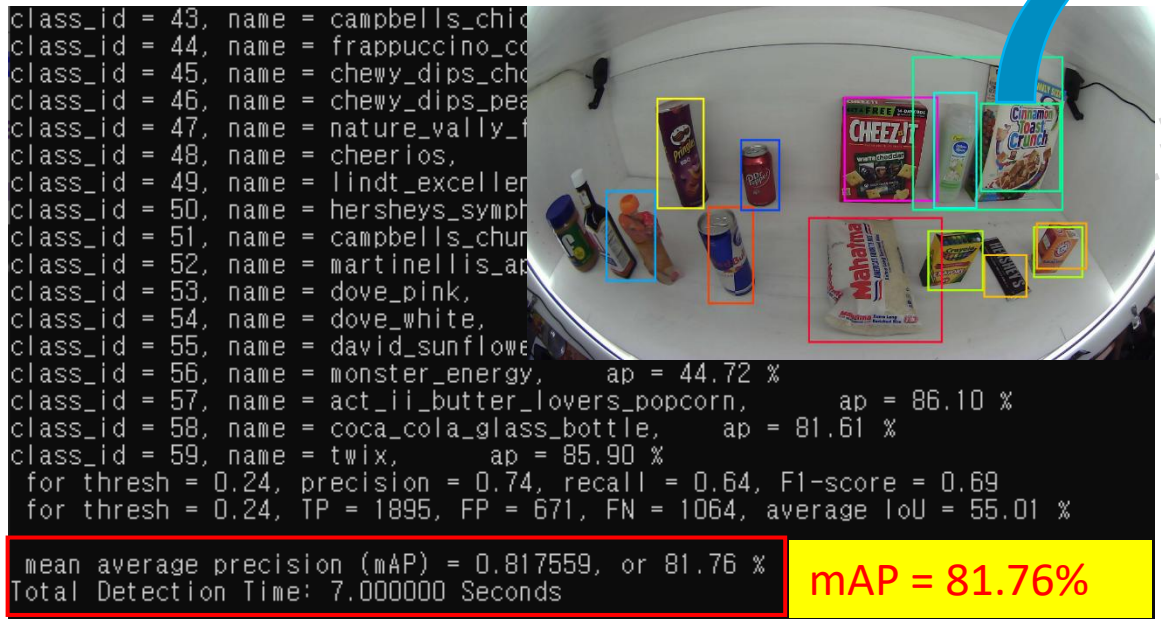
Copyright 2022-2026 (사)차세대반도체혁신공유대학 사업단) all rights reserved.

System Overview: Quantization (S/W)

- Quantization: Compute operations with integer operations (e.g., INT8 or UINT8)

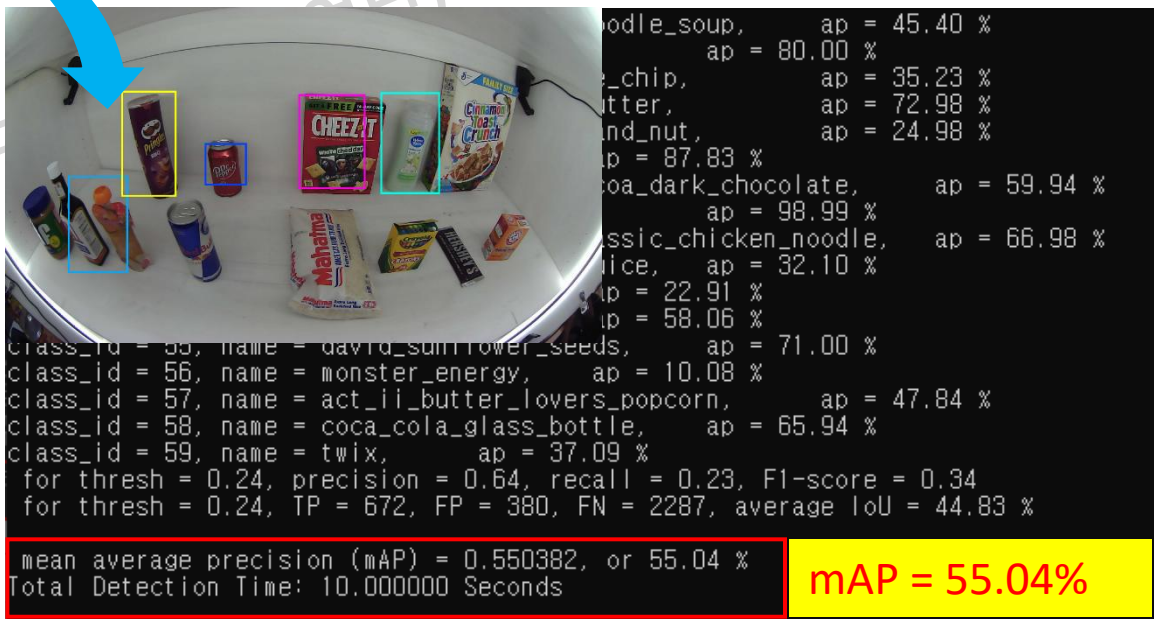
⇒ Must **improve the accuracy** of a quantized model

Full-Precision



mAP = 81.76%

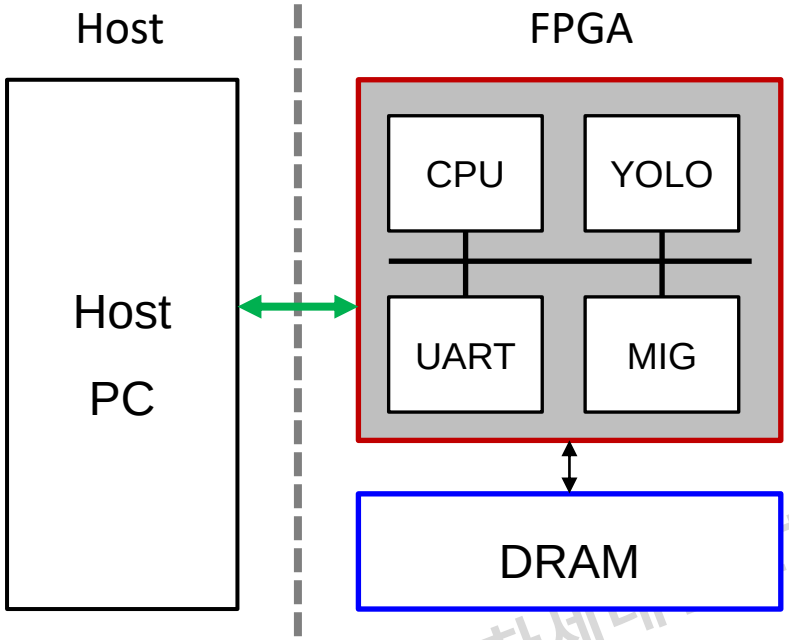
Sample: After Quantization (INT8)

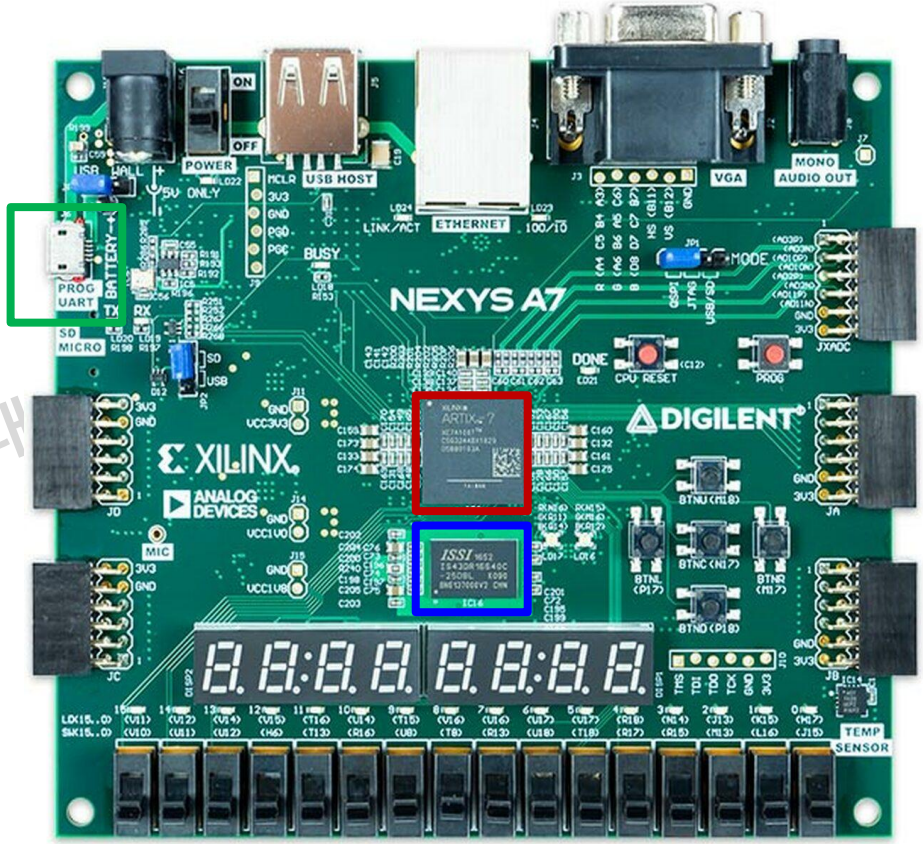


mAP = 55.04%

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

System Overview: Host-FPGA



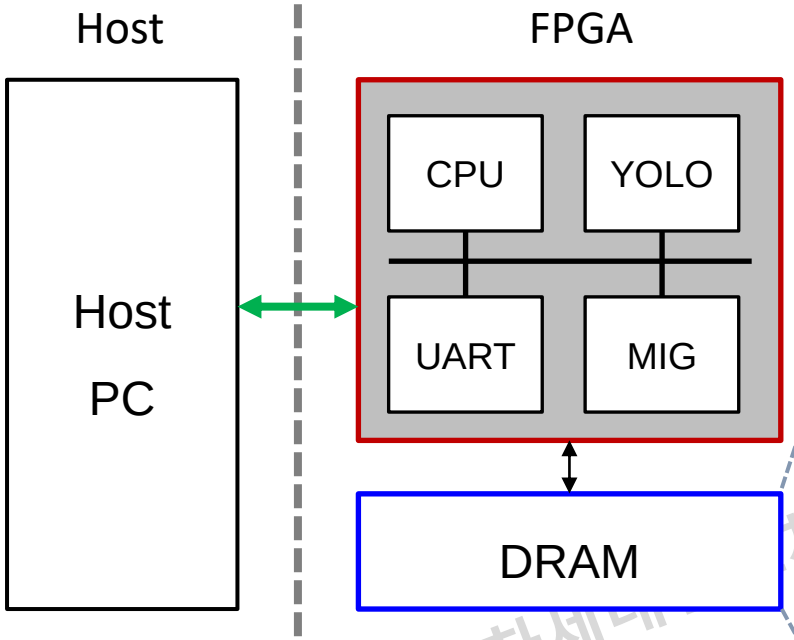


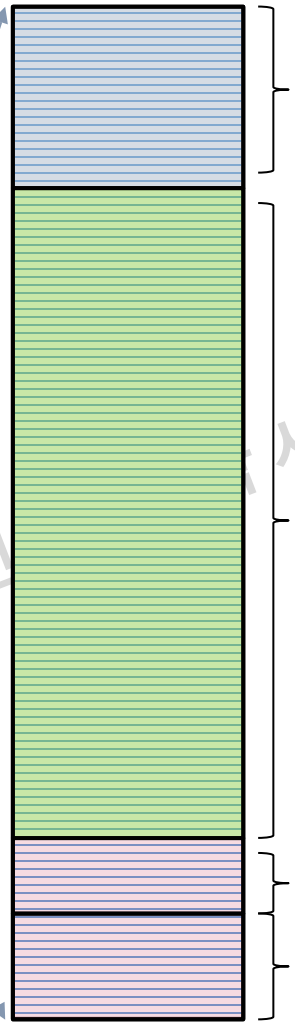
- The host and FPGA communicate via UART
- FPGA Side:
  - FPGA: CPU (MicroBlaze, UART, MIG, YOLO)
  - DDR2 DRAM

Copyright 2022-2026, (차세대반도체 혁신공유대학

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

System Overview: Memory Allocation





Input (256×256×4 = 256 KB)

Model parameters (~3.4 MB)
L0,2,4: 4096×72
L6: 3×3×64×128
L8: 3×3×128×256
L10: 3×3×256×512
L12: 1×1×512×256
L13: 3×3×256×512
L14: 1×1×512×196
L17: 1×1×256×128
L20: 1×1×384×196

Output 1 (8×8×196 = 12.5 KB)

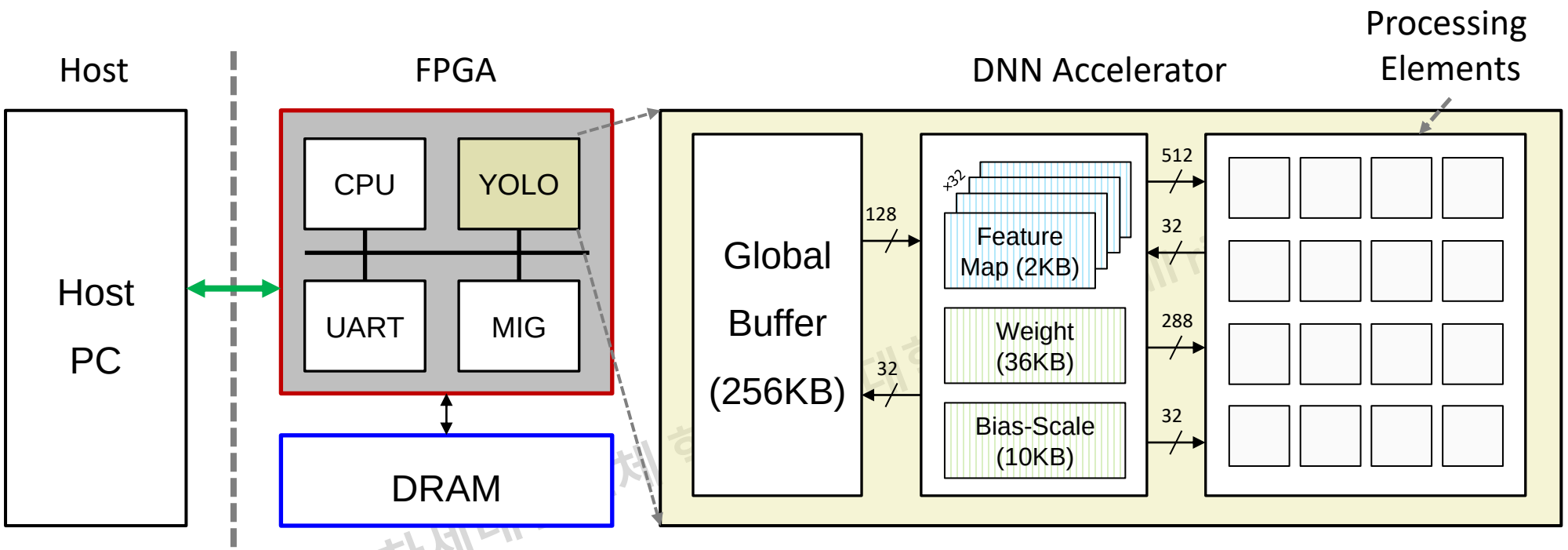
Output 2 (16×16×196 = 50.2 KB)

- DRAM is utilized to store:
  - Input, parameters, and outputs
  - Each is defined by start_addr & range

Copyright 2022-2026 (차세대반도체 혁신

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

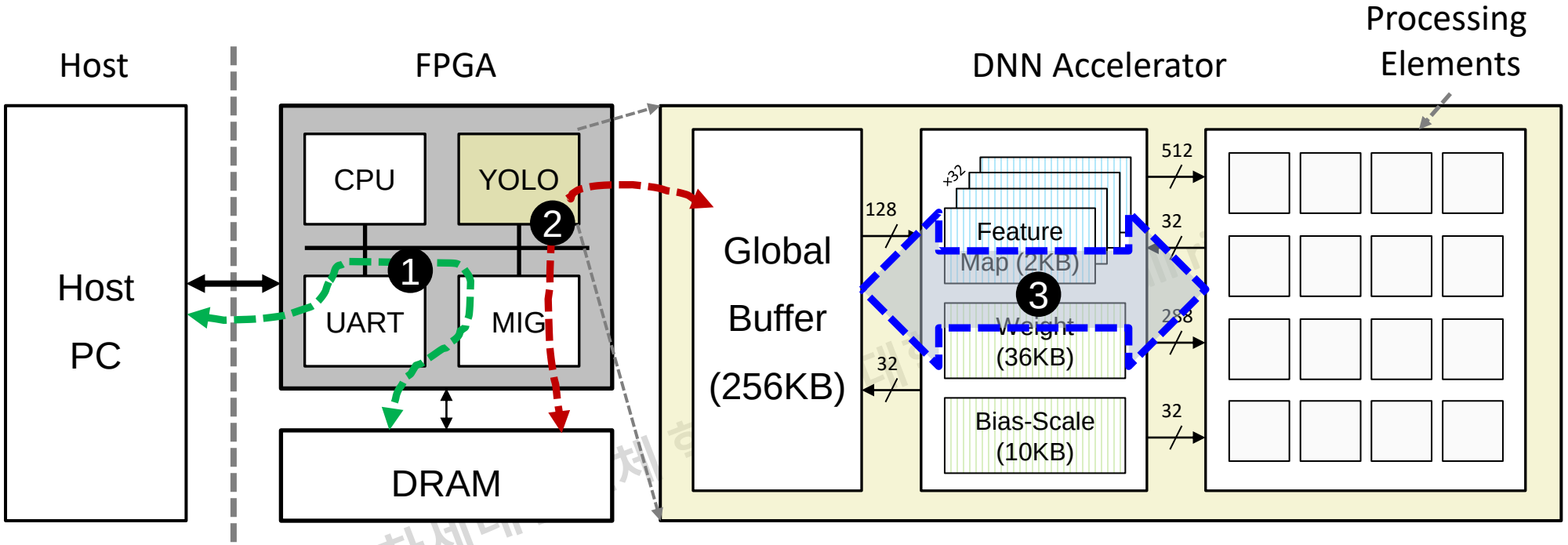
System Overview: DNN Accelerator Example



- DNN Accelerator (YOLO) includes:
  - L2 Global Buffer (256KB): Can store an input (256×256×4) or an output (128×128×16)
  - L1 Buffers: Dual F-Maps (32×2KB), Weight (4096×72 = 36KB), Bias-Scale (2560×32)
  - Can directly access DRAM for loading an input and weights and storing outputs.

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단)

System Overview: Data Movement



- Three main communication channels:
  - Host ⇔ DRAM on FPGA via UART
  - DRAM ⇔ L2 Global Buffer via DMA
  - L2 ⇔ L1 Local Buffers ⇔ PEs or MACs

Copyright 2022-2026 (차세대반도체 혁신공유대학 사업단)

Outlines

- Recap
- System Overview
- Test Vector Generation
- Sample Top Design

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

Quantized Model

- File: yolov2_forward_network_quantized.c
  - do_quantization(network net)
- Each quantized layer has two parameters:
  - weight_quant_multiplier
  - input_quant_multiplier
- **Task: To improve the accuracy, you must find a set of weight and input multipliers.**
- Here, we utilize the **default multipliers**.

```c
void do_quantization(network net) {
    int counter = 0;

    int j;
    //Dummy weight_quantization
    #define TOTAL_CALIB_LAYER 11
    // TODO
    //{{{
    float weight_quant_multiplier[TOTAL_CALIB_LAYER] = {
      16,     //conv 0
      64,     //conv 2
      64,     //conv 4
      64,     //conv 6
      64,     //conv 8
      64,     //conv 10
      64,     //conv 12
      64,     //conv 13
      64,     //conv 14
      64,     //conv 17
      64};    //conv 20

    float input_quant_multiplier[TOTAL_CALIB_LAYER] = {
     128,     //conv 0
      16,     //conv 2
      16,     //conv 4
      16,     //conv 6
      16,     //conv 8
      16,     //conv 10
      16,     //conv 12
      16,     //conv 13
      16,     //conv 14
      16,     //conv 17
      16};    //conv 20

    printf("Multipler    Input    Weight    Bias\n");
    for (j = 0; j < net.n; ++j) {
        layer *l = &net.layers[j];
```

Copyright 2022-2026. (차세대반도체 혁신공유

Quantized Model

- File: yolov2_forward_network_quantized.c → do_quantization(network net)

```c
    printf("Multipler    Input    Weight    Bias\n");
    for (j = 0; j < net.n; ++j) {
        layer *l = &net.layers[j];

        /* ... */

        //printf("\n");
        if (l->type == CONVOLUTIONAL) { // Quantize conv layer only
            size_t const filter_size = l->size*l->size*l->c;

            int i, fil;

            // Quantized Parameters
            //{{{
                // Input feature map
                l->input_quant_multiplier   = (counter < TOTAL_CALIB_LAYER) ? input_quant_multiplier [counter] : 16;

                // Weight
                l->weights_quant_multiplier = (counter < TOTAL_CALIB_LAYER) ? weight_quant_multiplier[counter] : 16;

                ++counter;
            //}}}
            // Weight Quantization
            for (fil = 0; fil < l->n; ++fil) {          //
                for (i = 0; i < filter_size; ++i) {
                    float w = l->weights[fil*filter_size + i] * l->weights_quant_multiplier; // Scale
                    l->weights_int8[fil*filter_size + i] = max_abs(w, MAX_VAL_8); // Clip
                }
            }

            // Bias Quantization
            float biases_multiplier = (l->weights_quant_multiplier * l->input_quant_multiplier);
            for (fil = 0; fil < l->n; ++fil) {
                float b = l->biases[fil] * biases_multiplier; // Scale
                l->biases_quant[fil] = max_abs(b, MAX_VAL_16); // Clip
            }

            //printf(" CONV%d multipliers: input %g, weights %g, bias %g \n", j, l->input_quant_multiplier, l->weights_quant_mul
            printf(" CONV%d: \t%g \t%g \t%g \n", j, l->input_quant_multiplier, l->weights_quant_multiplier, biases_multiplier);
        }
        else {
            //printf(" No quantization for layer %d (layer type: %d) \n", j, l->type);
        }
    }
}
```

For each **convolutional** layer

- Compute the size of a filter

  filter_size = l->size * l->size * l-> c

  Example:

  + **Layer 0:** l-> size = 3, l->c = 3

  → filter_size = 3 * 3 * 3 = 27

  + **Layer 2:** l-> size = 3, l-> c = 16

  → filter_size = 3 * 3 * 16 = 144

  + **Layer 4:** l-> size = 3, l-> c = 32

  → filter_size = 3 * 3 * 32 = 288

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Model Quantization

- File: yolov2_forward_network_quantized.c → do_quantization(network net)

```c
    printf("Multipler    Input    Weight    Bias\n");
    for (j = 0; j < net.n; ++j) {
        layer *l = &net.layers[j];

        /* ... */

        //printf("\n");
        if (l->type == CONVOLUTIONAL) { // Quantize conv layer only
            size_t const filter_size = l->size*l->size*l->c;

            int i, fil;

            // Quantized Parameters
            //{{{
                // Input feature map
                l->input_quant_multiplier   = (counter < TOTAL_CALIB_LAYER) ? input_quant_multiplier [counter] : 16;

                // Weight
                l->weights_quant_multiplier = (counter < TOTAL_CALIB_LAYER) ? weight_quant_multiplier[counter] : 16;

                ++counter;
            //}}}
            // Weight Quantization
            for (fil = 0; fil < l->n; ++fil) {          //
                for (i = 0; i < filter_size; ++i) {
                    float w = l->weights[fil*filter_size + i] * l->weights_quant_multiplier; // Scale
                    l->weights_int8[fil*filter_size + i] = max_abs(w, MAX_VAL_8); // Clip
                }
            }

            // Bias Quantization
            float biases_multiplier = (l->weights_quant_multiplier * l->input_quant_multiplier);
            for (fil = 0; fil < l->n; ++fil) {
                float b = l->biases[fil] * biases_multiplier; // Scale
                l->biases_quant[fil] = max_abs(b, MAX_VAL_16); // Clip
            }

            //printf(" CONV%d multipliers: input %g, weights %g, bias %g \n", j, l->input_quant_multiplier, l->weights_quant_mul
            printf(" CONV%d: \t%g \t%g \t%g \n", j, l->input_quant_multiplier, l->weights_quant_multiplier, biases_multiplier);
        }
        else {
            //printf(" No quantization for layer %d (layer type: %d) \n", j, l->type);
        }
    }
}
```

For each **convolutional** layer

- Set the multipliers from the arrays based on the layer counter
- No need to change this code

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Model Quantization

- File: yolov2_forward_network_quantized.c → do_quantization(network net)

```c
    printf("Multipler    Input    Weight    Bias\n");
    for (j = 0; j < net.n; ++j) {
        layer *l = &net.layers[j];

        /* ... */

        //printf("\n");
        if (l->type == CONVOLUTIONAL) { // Quantize conv layer only
            size_t const filter_size = l->size*l->size*l->c;

            int i, fil;

            // Quantized Parameters
            //{{{
                // Input feature map
                l->input_quant_multiplier   = (counter < TOTAL_CALIB_LAYER) ? input_quant_multiplier [counter] : 16;

                // Weight
                l->weights_quant_multiplier = (counter < TOTAL_CALIB_LAYER) ? weight_quant_multiplier[counter] : 16;

                ++counter;
            //}}}
            // Weight Quantization
            for (fil = 0; fil < l->n; ++fil) {          //
                for (i = 0; i < filter_size; ++i) {
                    float w = l->weights[fil*filter_size + i] * l->weights_quant_multiplier; // Scale
                    l->weights_int8[fil*filter_size + i] = max_abs(w, MAX_VAL_8); // Clip
                }
            }

            // Bias Quantization
            float biases_multiplier = (l->weights_quant_multiplier * l->input_quant_multiplier);
            for (fil = 0; fil < l->n; ++fil) {
                float b = l->biases[fil] * biases_multiplier; // Scale
                l->biases_quant[fil] = max_abs(b, MAX_VAL_16); // Clip
            }

            //printf(" CONV%d multipliers: input %g, weights %g, bias %g \n", j, l->input_quant_multiplier, l->weights_quant_mul
            printf(" CONV%d: \t%g \t%g \t%g \n", j, l->input_quant_multiplier, l->weights_quant_multiplier, biases_multiplier);
        }
        else {
            //printf(" No quantization for layer %d (layer type: %d) \n", j, l->type);
        }
    }
}
```

For each **convolutional** layer

- For each filter index (fil)
  - Convert the full-precision element in a filter into an INT8 weight

- Note: filter indexing
  - To access an ith-element in a filter fil, we use the index (fil * filter_size + i)
  - Example:
    - Layer 0: filter_size = 27
      + 0~26: 1st filter 3x3x3
      + 27~53: 2nd filter 3x3x3
      + …
      + 405~431: 16th filter 3x3x3

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Model Quantization: Saving a quantized model

- File: yolov2_forward_network_quantized.c → `save_quantized_model(network net)`

```c
378 void save_quantized_model(network net) {
379     int j;
380     for (j = 0; j < net.n; ++j) {
381         layer* l = &net.layers[j];
382         if (l->type == CONVOLUTIONAL) {
383             size_t filter_size = l->size * l->size * l->c;
384
385             printf(" Saving quantized weights, bias, and scale for CONV%02d \n", j);
386
387             char weightfile [100];
388             char biasfile   [100];
389             char scalefile  [100];
390
391             sprintf(weightfile  , "C:/skeleton/bin/log_param/CONV%02d_param_weight.hex", j);
392             sprintf(biasfile    , "C:/skeleton/bin/log_param/CONV%02d_param_biases.hex", j);
393             sprintf(scalefile   , "C:/skeleton/bin/log_param/CONV%02d_param_scales.hex", j);
394             FILE* fp_w = fopen(weightfile, "w");
395             FILE* fp_b = fopen(biasfile, "w");
396             FILE* fp_s = fopen(scalefile, "w");
397
398             int f;
399             for (f = 0; f < l->n; f++) {    // Out_channel
400                 // Weights
401                 //{{{
402                 for (int i = 0; i < filter_size; ++i) {     // Filter size
403                     int w_index = f * filter_size + i;
404                     fprintf(fp_w, "%02x\n", (uint8_t) l->weights_int8[w_index]);
405                 }
406                 //}}}
```

For each **convolutional** layer

- Open three new files for weights, biases, and scales
- Similarly, compute the size of a filter

  filter_size = l->size * l->size * l-> c

  Example:

  + **Layer 0:** l-> size = 3, l->c = 3

  → filter_size = 3 * 3 * 3 = 27

  + **Layer 2:** l-> size = 3, l-> c = 16

  → filter_size = 3 * 3 * 16 = 144

  + **Layer 4:** l-> size = 3, l-> c = 32

  → filter_size = 3 * 3 * 32 = 288

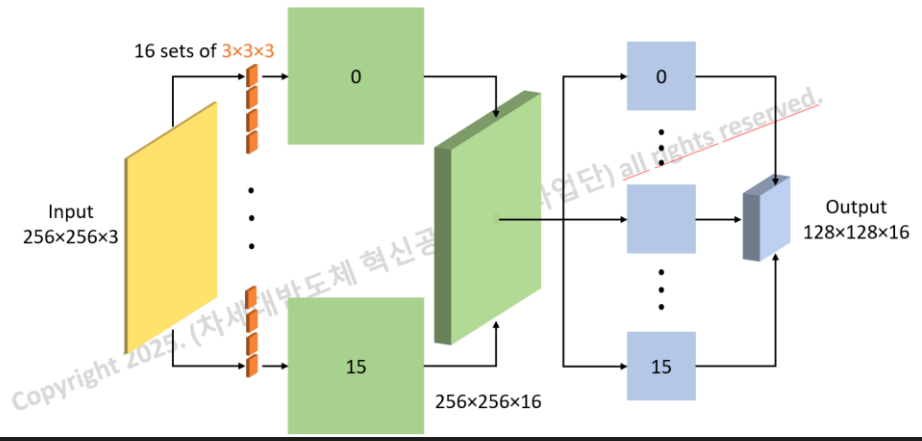
Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Model Quantization: Saving a quantized model

- File: yolov2_forward_network_quantized.c → `save_quantized_model(network net)`

```c
int f;
for (f = 0; f < l->n; f++) {    // Out_channel
    // Weights
    //{{{
    for (int i = 0; i < filter_size; ++i) {     // Filter size
        int w_index = f * filter_size + i;
        fprintf(fp_w, "%02x\n", (uint8_t) l->weights_int8[w_index]);
    }
    //}}}

    // Biases
    //{{{
```

Convolution + max-pooling: Layers 0, 1



For each **convolutional** layer

- Outer loop: Filter
  - For (f = 0; f < l->n; f++)
- Inner loop: Element in a filter
  - For (i = 0; i < filter_size; i++)
- Note: Indexing: (fil * filter_size + i))
  [fil_row][fil_col][in_chn][out_chn]
- Example: Layer 00 (filter_size = 3x3x3)

0~26: 1st filter
w[0][0][0][0] = 13 (Line 1)
w[0][1][0][0] = f5 (Line 2)
w[0][2][0][0] = fa (Line 3)
....
w[2][2][2][0] = fb (Line 27)

27~53: 2nd filter
w[0][0][0][1] = 0c (Line 28)
....

C: > skeleton > bin > log_param >

```
1  13
2  f5
3  fa
4  3d
5  ef
6  d6
7  17
8  fe
9  eb
10 11
11 f4
12 fb
13 30
14 f2
15 e0
16 0b
17 fc
18 fd
19 10
20 f0
21 00
22 31
23 ef
24 e1
25 06
26 ff
27 fb
28 0c
```

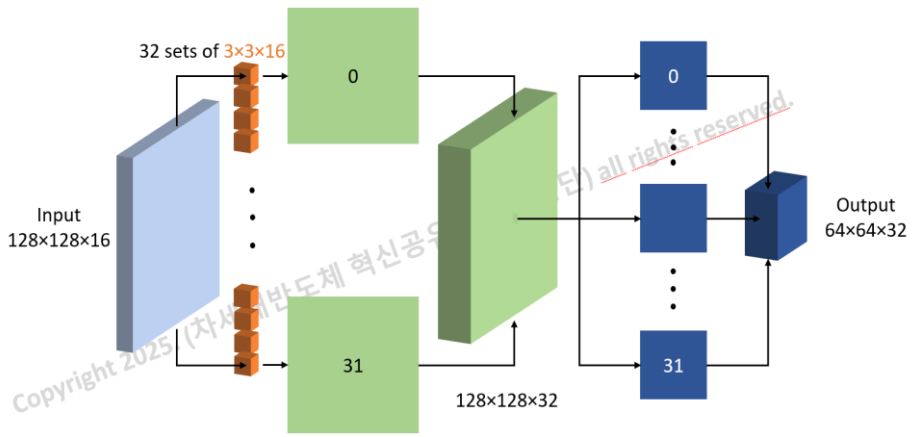
Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Model Quantization: Saving a quantized model

- File: yolov2_forward_network_quantized.c → `save_quantized_model(network net)`

```c
int f;
for (f = 0; f < l->n; f++) {    // Out_channel
    // Weights
    //{{{
    for (int i = 0; i < filter_size; ++i) {     // Filter size
        int w_index = f * filter_size + i;
        fprintf(fp_w, "%02x\n", (uint8_t) l->weights_int8[w_index]);
    }
    //}}}

    // Biases
    //{{{
```

Convolution + max-pooling: Layers 2, 3



For each **convolutional** layer

- Outer loop: Filter
  - For (f = 0; f < l->n; f++)
- Inner loop: Element in a filter
  - For (i = 0; i < filter_size; i++)
- Note: Indexing: (fil * filter_size + i))
  [fil_row][fil_col][in_chn][out_chn]
- Example: Layer 02 (filter_size = 3x3x16)

0~143: 1st filter
w[0][0][0][0] = ff (Line 1)
w[0][1][0][0] = ff (Line 2)
w[0][2][0][0] = 04 (Line 3)
....
w[2][2][15][0] = 00 (Line 144)

144~287: 2nd filter
w[0][0][0][1] = 03 (Line 145)
...

C: > skeleton > bin > log_param >

```
1  ff
2  ff
3  04
4  ff
5  f8
6  00
7  04
8  fe
9  01
10 00
11 fc
12 00
13 00
14 fa
15 00
16 06
17 00
18 05
19 00
20 fe
21 03
22 ff
23 ff
24 01
25 ff
26 01
27 00
28 05
```

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Model Quantization: Saving a quantized model

- File: yolov2_forward_network_quantized.c → `save_quantized_model(network net)`

```c
int f;
for (f = 0; f < l->n; f++) {    // Out_channel
    // Weights
    //{{{
    for (int i = 0; i < filter_size; ++i) {     // Filter size
        int w_index = f * filter_size + i;
        fprintf(fp_w, "%02x\n", (uint8_t) l->weights_int8[w_index]);
    }
    //}}}

    // Biases
    //{{{
    fprintf(fp_b, "%04x\n", (uint16_t) l->biases_quant[f]);
    //}}}

    // Dequantization or Scaling
    //{{{
    // Find the input quantization factor for the next CONV layer
    int next_input_quant_multiplier = 1;
    for (int z = l->index+ 1; z < net.n; ++z) {
        if (net.layers[z].type == CONVOLUTIONAL) {
            next_input_quant_multiplier = net.layers[z].input_quant_multiplier;
            break;
        }
    }

    int scale = (l->input_quant_multiplier * l->weights_quant_multiplier) / next_input_quant_multiplier;
    fprintf(fp_s, "%04x\n", (uint16_t) scale);
    //}}}
}
```

Layer 00

C: > skeleton > bin

1	02c2
2	ffc0
3	0791
4	fd03
5	0ab8
6	0c44
7	0053
8	04c5
9	097a
10	e7f0
11	0763
12	06bb
13	03e7
14	044b
15	01bb
16	f6e6
17	

Layer 02

C: > skeleton > bin

1	08de
2	0698
3	04de
4	04ed
5	0379
6	0116
7	0189
8	05b8
9	03d0
10	0117
11	f77b
12	ffc2
13	0324
14	fe2e
15	0556
16	ff28
17	09c3

Layer 04

C: > skeleton > bin

1	0a72
2	023a
3	08b4
4	0a68
5	054a
6	0422
7	01a5
8	023b
9	04de
10	ff1a
11	fcc7
12	005b
13	048d
14	fd2c
15	fb2b
16	040b
17	01e2

Import note: Each filter has one bias.
We may store all biases in a ROM file.

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Model Quantization: Saving a quantized model

- File: yolov2_forward_network_quantized.c → `save_quantized_model(network net)`

```c
int f;
for (f = 0; f < l->n; f++) {    // Out_channel
    // Weights
    //{{{
    for (int i = 0; i < filter_size; ++i) {     // Filter size
        int w_index = f * filter_size + i;
        fprintf(fp_w, "%02x\n", (uint8_t) l->weights_int8[w_index]);
    }
    //}}}

    // Biases
    //{{{
    fprintf(fp_b, "%04x\n", (uint16_t) l->biases_quant[f]);
    //}}}

    // Dequantization or Scaling
    //{{{
    // Find the input quantization factor for the next CONV layer
    int next_input_quant_multiplier = 1;
    for (int z = l->index+ 1; z < net.n; ++z) {
        if (net.layers[z].type == CONVOLUTIONAL) {
            next_input_quant_multiplier = net.layers[z].input_quant_multiplier;
            break;
        }
    }

    int scale = (l->input_quant_multiplier * l->weights_quant_multiplier) / next_input_quant_multiplier;
    fprintf(fp_s, "%04x\n", (uint16_t) scale);
    //}}}
}
```

Layer 00

C: > skeleton > bin

1	0080
2	0080
3	0080
4	0080
5	0080
6	0080
7	0080
8	0080
9	0080
10	0080
11	0080
12	0080
13	0080
14	0080
15	0080
16	0080
17	

Layer 02

C: > skeleton > bin

1	0040
2	0040
3	0040
4	0040
5	0040
6	0040
7	0040
8	0040
9	0040
10	0040
11	0040
12	0040
13	0040
14	0040
15	0040
16	0040
17	0040

Layer 04

C: > skeleton > bin

1	0040
2	0040
3	0040
4	0040
5	0040
6	0040
7	0040
8	0040
9	0040
10	0040
11	0040
12	0040
13	0040
14	0040
15	0040
16	0040
17	0040

Import note: Each layer has one scale
We may use registers to store them

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Activation Quantization: Input (Software)

- Activation quantization (void forward_convolutional_layer_q(network net, layer l, network_state s))
  - input_uint8 ← max_abs(input * input_quant_multiplier, MAX_VAL_8)

INT8/UINT8

Compute input_uint8
max(input*input_quant_multiplier, 255)

Layer #0

Convert input_uint8 and weights_int8 into matrices (im2col_cpu_int8)

Do matrix multiplication (**gemm_nn_int8_int32**)

Do bias addition (+ biases_quant)

Do activation (RELU: f(x) = max(0,x))

Do descaling or de-quantization (divide by (input_quant_multiplier* weights_quant_multiplier))

Compute input_uint8 (Layer#2)
max(input*input_quant_multiplier, 255)

INT8/UINT8

```
void forward_convolutional_layer_q(network net, layer l, network_state state)
{

    int out_h = (l.h + 2 * l.pad - l.size) / l.stride + 1;    // output_height=i
    int out_w = (l.w + 2 * l.pad - l.size) / l.stride + 1;    // output_width=in
    int i, j;
    int const out_size = out_h*out_w;

    typedef int32_t conv_t;    // l.output
    conv_t *output_q = calloc(l.outputs, sizeof(conv_t));

    state.input_uint8 = (int8_t*)calloc(l.inputs, sizeof(uint8_t)); //state.inpu
    int z;
    for (z = 0; z < l.inputs; ++z) {
        int16_t src = state.input[z] * l.input_quant_multiplier;
        state.input_uint8[z] = max_abs(src, MAX_VAL_UINT_8);    //state.input_in
    }

```

Copyright 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Activation Quantization: Input (Software)

- Activation quantization (void forward_convolutional_layer_q(network net, layer l, network_state s))
  - input_uint8 ← max_abs(input * input_quant_multiplier, MAX_VAL_8)

INT8/UINT8

Compute input_uint8
max(input*input_quant_multiplier, 255)

Layer #0

Convert input_uint8 and weights_int8 into matrices (im2col_cpu_int8)

Do matrix multiplication (**gemm_nn_int8_int32**)

Do bias addition (+ biases_quant)

Do activation (RELU: f(x) = max(0,x))

Do descaling or de-quantization (divide by (input_quant_multiplier* weights_quant_multiplier))

Compute input_uint8 (Layer#2)
max(input*input_quant_multiplier, 255)

INT8/UINT8

```
// Write data for the HW verification
//{{{
if (run_single_image_test) {
    // Input Feature Map (IFM)
    char file_input_femap[100];
    snprintf(file_input_femap, sizeof(file_input_femap), "C:/skeleton/bin/log_feamap/CONV%02d_input.hex"
    FILE* fp = fopen(file_input_femap, "w");

    // Data Format: [Channel, Width, Height]
    for (int chn = 0; chn < l.c; chn++) {                    // IFM: Channel/index of an feature map
        for (int idx = 0; idx < l.h * l.w; idx++) {          // IFM: Pixel index in ONE feature map
            int i = chn * l.h * l.w + idx;                   // IFM: Pixel index
            uint8_t pixel = state.input_uint8[i];
            fprintf(fp, "%02x\n", pixel);
        }
    }
    if (fp) fclose(fp);
}
//}}}
```

Note: we can store an input image (input_uint8) in a file
Example: Layer 0: Input[256][256][3], Layer 2: [128][128][16]
- Default: Outer loop: Channel index, inner loop: Pixel index
- You may adjust it for your preference data format

Spring 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Activation Quantization: Input (Software)

- Activation quantization (void forward_convolutional_layer_q(network net, layer l, network_state s))
  - input_uint8 ← max_abs(input * input_quant_multiplier, MAX_VAL_8)

INT8/UINT8

Compute input_uint8
max(input*input_quant_multiplier, 255)

Layer #0

Convert input_uint8 and weights_int8 into matrices (im2col_cpu_int8)

Do matrix multiplication (**gemm_nn_int8_int32**)

Do bias addition (+ biases_quant)

Do activation (RELU: f(x) = max(0,x))

Do descaling or de-quantization (divide by (input_quant_multiplier* weights_quant_multiplier))

Compute input_uint8 (Layer#2)
max(input*input_quant_multiplier, 255)

INT8/UINT8

```
// Convolution
int m = l.n;
int k = l.size*l.size*l.c;
int n = out_h*out_w;
int8_t *a = l.weights_int8;
uint8_t* b = (uint8_t*)state.workspace; //int8_t *b = (int8_t *)state.workspace;
conv_t *c = output_q;    // int32_t

// Use GEMM (as part of BLAS)
im2col_cpu_int8(state.input_uint8, l.c, l.h, l.w, l.size, l.stride, l.pad, b);

int t;    // multi-thread gemm
#pragma omp parallel for
for (t = 0; t < m; ++t) {
    gemm_nn_int8_int32(1, n, k, 1, a + t*k, k, b, n, c + t*n, n);
}
free(state.input_uint8);    //free(state.input_int8);

// Bias addition
int fil;
for (fil = 0; fil < l.n; ++fil) {
    for (j = 0; j < out_size; ++j) {
        output_q[fil*out_size + j] = output_q[fil*out_size + j] + l.biases_quant[fil];
    }
```

Do convolution using a general matrix multiplication (GEMM) format (Don't touch).
output_q ← convolve (weights_int8, state)

Copyright 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Activation Quantization: Input (Software)

- Activation quantization (void forward_convolutional_layer_q(network net, layer l, network_state s))
  - input_uint8 ← max_abs(input * input_quant_multiplier, MAX_VAL_8)

INT8/UINT8

Compute input_uint8
max(input*input_quant_multiplier, 255)

Layer #0

Convert input_uint8 and weights_int8 into matrices (im2col_cpu_int8)

Do matrix multiplication (**gemm_nn_int8_int32**)

Do bias addition (+ biases_quant)

Do activation (RELU: f(x) = max(0,x))

Do descaling or de-quantization (divide by (input_quant_multiplier* weights_quant_multiplier))

Compute input_uint8 (Layer#2)
max(input*input_quant_multiplier, 255)

INT8/UINT8

```
// Convolution
int m = l.n;
int k = l.size*l.size*l.c;
int n = out_h*out_w;
int8_t *a = l.weights_int8;
uint8_t* b = (uint8_t*)state.workspace; //int8_t *b = (int8_t *)state.workspace;
conv_t *c = output_q;    // int32_t

// Use GEMM (as part of BLAS)
im2col_cpu_int8(state.input_uint8, l.c, l.h, l.w, l.size, l.stride, l.pad, b);

int t;    // multi-thread gemm
#pragma omp parallel for
for (t = 0; t < m; ++t) {
    gemm_nn_int8_int32(1, n, k, 1, a + t*k, k, b, n, c + t*n, n);
}
free(state.input_uint8);    //free(state.input_int8);

// Bias addition
int fil;
for (fil = 0; fil < l.n; ++fil) {
    for (j = 0; j < out_size; ++j) {
        output_q[fil*out_size + j] = output_q[fil*out_size + j] + l.biases_quant[fil];
    }
}
```

For each filter (fil), add its corresponding bias
Index: fil

Copyright 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Activation Quantization: Input (Software)

- Activation quantization (void forward_convolutional_layer_q(network net, layer l, network_state s))
  - input_uint8 ← max_abs(input * input_quant_multiplier, MAX_VAL_8)

INT8/UINT8

Compute input_uint8
max(input*input_quant_multiplier, 255)

Layer #0

Convert input_uint8 and weights_int8 into matrices (im2col_cpu_int8)

Do matrix multiplication (**gemm_nn_int8_int32**)

Do bias addition (+ biases_quant)

Do activation (RELU: f(x) = max(0,x))

Do descaling or de-quantization (divide by (input_quant_multiplier* weights_quant_multiplier))

Compute input_uint8 (Layer#2)
max(input*input_quant_multiplier, 255)

INT8/UINT8

```
// Convolution
int m = l.n;
int k = l.size*l.size*l.c;
int n = out_h*out_w;
int8_t *a = l.weights_int8;
uint8_t* b = (uint8_t*)state.workspace; //int8_t *b = (int8_t *)state.workspace;
conv_t *c = output_q;    // int32_t

// Use GEMM (as part of BLAS)
im2col_cpu_int8(state.input_uint8, l.c, l.h, l.w, l.size, l.stride, l.pad, b);

int t;    // multi-thread gemm
#pragma omp parallel for
for (t = 0; t < m; ++t) {
    gemm_nn_int8_int32(1, n, k, 1, a + t*k, k, b, n, c + t*n, n);
}
free(state.input_uint8);    //free(state.input_int8);

// Bias addition
int fil;
for (fil = 0; fil < l.n; ++fil) {
    for (j = 0; j < out_size; ++j) {
        output_q[fil*out_size + j] = output_q[fil*out_size + j] + l.biases_quant[fil];
    }
}

// Activation
if (l.activation == RELU) {
    for (i = 0; i < l.n * out_size; ++i) {
        output_q[i] = (output_q[i] > 0) ? output_q[i] : 0;
    }
}
```

Copyright 2026, 차세대반도체 혁신공유대학 사업단. All rights reserved.

Activation Quantization: De-Scaling

- Activation quantization (void forward_convolutional_layer_q(network net, layer l, network_state s))
  - input_uint8 ← max_abs(input * input_quant_multiplier, MAX_VAL_8)

INT8/UINT8

Compute input_uint8
max(input*input_quant_multiplier, 255)

Layer #0

Convert input_uint8 and weights_int8 into matrices (im2col_cpu_int8)

Do matrix multiplication (**gemm_nn_int8_int32**)

Do bias addition (+ biases_quant)

Do activation (RELU: f(x) = max(0,x))

Do descaling or de-quantization (divide by (input_quant_multiplier* weights_quant_multiplier))

Compute input_uint8 (Layer#2)
max(input*input_quant_multiplier, 255)

INT8/UINT8

```
// De-scaling or De-quantization
float ALPHA1 = 1 / (l.input_quant_multiplier * l.weights_quant_multiplier);
for (i = 0; i < l.outputs; ++i) {
    l.output[i] = output_q[i] * ALPHA1;
}


// Write data for the HW verification
//{{{
if (run_single_image_test) {
    // Output Feature Map (OFM)
    int z;
    int next_input_quant_multiplier = 1;
    for (z = state.index + 1; z < net.n; ++z) {
        if (net.layers[z].type == CONVOLUTIONAL) {
            next_input_quant_multiplier = net.layers[z].input_quant_multiplier;
            break;
        }
    }
    char file_output_femap[100];
    snprintf(file_output_femap, sizeof(file_output_femap), "C:/skeleton/bin/log_feamap/CONV%02d_output.hex",
    FILE* fp = fopen(file_output_femap, "w");
    // Data Format: [Channel, Width, Height]
    for (int chn = 0; chn < l.n; chn++) {           // OFM: Channel/index of an feature map
        for (int idx = 0; idx < out_size; idx++) {  // OFM: Pixel index in ONE feature map
            int i = chn * out_size + idx;           // OFM: Pixel index
            int16_t src = l.output[i] * next_input_quant_multiplier;
            uint8_t pixel = max_abs(src, MAX_VAL_UINT_8);
            fprintf(fp, "%02x\n", pixel);
        }
    }
    if (fp) fclose(fp);
}
```

Find input_quant_multiplier of the next layer

Copyright

Activation Quantization

- Activation quantization (void forward_convolutional_layer_q(network net, layer l, network_state s))
  - input_uint8 ← max_abs(input * input_quant_multiplier, MAX_VAL_8)

Layer 00: 256x256x3 → 256x256x16 (Before Pooling)

CONV00_input.hex | CONV00_output.hex

C: > skeleton > bin > log_feamap > | C: > skeleton > bin > lo

196592	0b	1048562	00
196593	0d	1048563	00
196594	09	1048564	00
196595	08	1048565	00
196596	07	1048566	00
196597	06	1048567	00
196598	04	1048568	00
196599	05	1048569	00
196600	07	1048570	00
196601	04	1048571	00
196602	05	1048572	00
196603	05	1048573	00
196604	07	1048574	00
196605	07	1048575	00
196606	0d	1048576	00
196607	07	1048577	
196608	08		
196609			

Layer 02: 128x128x16 → 128x128x32 (Before Pooling)

CONV02_input.hex | CONV02_output.hex

map > test01 > CONV02_input.hex | map > test01 > CONV02

262131	00	524276	17
262132	01	524277	00
262133	03	524278	0b
262134	00	524279	1d
262135	00	524280	00
262136	00	524281	08
262137	00	524282	06
262138	00	524283	06
262139	00	524284	03
262140	00	524285	03
262141	00	524286	05
262142	00	524287	03
262143	00	524288	00
262144	00	524289	
262145			

Layer 02: 64x64x32 → 64x64x64 (Before Pooling)

CONV04_input.hex | CONV04_output.hex

map > test01 > CONV04_input.h | map > test01 > CONV0

131058	04	262130	00
131059	02	262131	00
131060	04	262132	00
131061	1e	262133	00
131062	46	262134	00
131063	39	262135	00
131064	25	262136	00
131065	0f	262137	03
131066	17	262138	00
131067	0b	262139	00
131068	1d	262140	00
131069	08	262141	00
131070	06	262142	05
131071	05	262143	07
131072	04	262144	00
131073		262145	

차세대반도체 혁신공유대학 사업단 all rights reserved.

Example: Loading data files

- Example: Configure a parameter to test Layer 00 (Note: test vectors are generated from S/W)

```
module tb_layer_00_no_pool;
parameter  CLK_PERIOD        = 10;   //100MHz
// FIXED ME
//{{{
parameter  LAYER_IFM_FILE    = "C:/skeleton/bin/log_feamap/CONV00_input.hex";
parameter  LAYER_OFM_FILE    = "C:/skeleton/bin/log_feamap/CONV00_output.hex";
parameter  LAYER_WGT_FILE    = "C:/skeleton/bin/log_param/CONV00_param_weight.hex";
parameter  LAYER_BIASES_FILE = "C:/skeleton/bin/log_param/CONV00_param_biases.hex";
parameter  LAYER_SCALES_FILE = "C:/skeleton/bin/log_param/CONV00_param_scales.hex";
parameter  IFM_WIDTH         = 256;
parameter  IFM_HEIGHT        = 256;
parameter  IFM_CHANNEL       = 3;
parameter  OFM_WIDTH         = 256; // WARNING: Disable pooling
parameter  OFM_HEIGHT        = 256; // WARNING: Disable pooling
parameter  OFM_CHANNEL       = 16;
parameter  FX                = 3;
parameter  FY                = 3;
parameter  MAX_POOL_STRIDE   = 2;
parameter  CONV_MODE         = 0; // 0: conv3x3, 1: conv1x1
parameter  CONV3x3_MODE      = 0;
parameter  CONV1x1_MODE      = 1;
parameter  MAX_POOL_ENA      = 0;   // WARNING: Disable pooling
parameter  ACT_RELU_ENA      = 1;
parameter  SCALE_SHIFT       = 7;   //conv00: 7, others: 6
    localparam FSIZE         = FX*FY*IFM_CHANNEL;
```

Copyright 2022 ... hts reserved.

Example: Loading data files

- Example: Configure a parameter to test Layer 02 (Note: test vectors are generated from S/W)

```
module tb_layer_02_no_pool;
parameter  CLK_PERIOD        = 10;   //100MHz
// FIXED ME
//{{{
parameter  LAYER_IFM_FILE    = "C:/skeleton/bin/log_feamap/CONV02_input.hex";
parameter  LAYER_OFM_FILE    = "C:/skeleton/bin/log_feamap/CONV02_output.hex";
parameter  LAYER_WGT_FILE    = "C:/skeleton/bin/log_param/CONV02_param_weight.hex";
parameter  LAYER_BIASES_FILE = "C:/skeleton/bin/log_param/CONV02_param_biases.hex";
parameter  LAYER_SCALES_FILE = "C:/skeleton/bin/log_param/CONV02_param_scales.hex";
parameter  IFM_WIDTH         = 128;
parameter  IFM_HEIGHT        = 128;
parameter  IFM_CHANNEL       = 16;
parameter  OFM_WIDTH         = 256; // WARNING: Disable pooling
parameter  OFM_HEIGHT        = 256; // WARNING: Disable pooling
parameter  OFM_CHANNEL       = 32;
parameter  FX                = 3;
parameter  FY                = 3;
parameter  MAX_POOL_STRIDE   = 2;
parameter  CONV_MODE         = 0; // 0: conv3x3, 1: conv1x1
parameter  CONV3x3_MODE      = 0;
parameter  CONV1x1_MODE      = 1;
parameter  MAX_POOL_ENA      = 0;   // WARNING: Disable pooling
parameter  ACT_RELU_ENA      = 1;
parameter  SCALE_SHIFT       = 6;   //conv00: 7, others: 6
    localparam FSIZE         = FX*FY*IFM_CHANNEL;
```

Copyright 2022 ... ts reserved.

Example: Loading data files

- Define a test vector for a layer (wgt_mem, ifm_mem, ofm_mem, biases, scales)

```
// Test data
reg [ 7:0] wgt_mem[0:FX*FY*IFM_CHANNEL*OFM_CHANNEL     -1]; // Filter
reg [ 7:0] ifm_mem[0:IFM_HEIGHT*IFM_WIDTH*IFM_CHANNEL  -1]; // Infmap
reg [ 7:0] ofm_mem[0:OFM_HEIGHT*OFM_WIDTH*OFM_CHANNEL  -1]; // Outfmap
reg [15:0] biases [0:OFM_CHANNEL                       -1];
reg [15:0] scales [0:OFM_CHANNEL                       -1];
```

- Load test vectors from files

```
initial begin: PROC_SimmemLoad
$readmemh(LAYER_WGT_FILE   , wgt_mem);
$readmemh(LAYER_IFM_FILE   , ifm_mem);
$readmemh(LAYER_OFM_FILE   , ofm_mem);
$readmemh(LAYER_BIASES_FILE, biases );
$readmemh(LAYER_SCALES_FILE, scales );
end
```

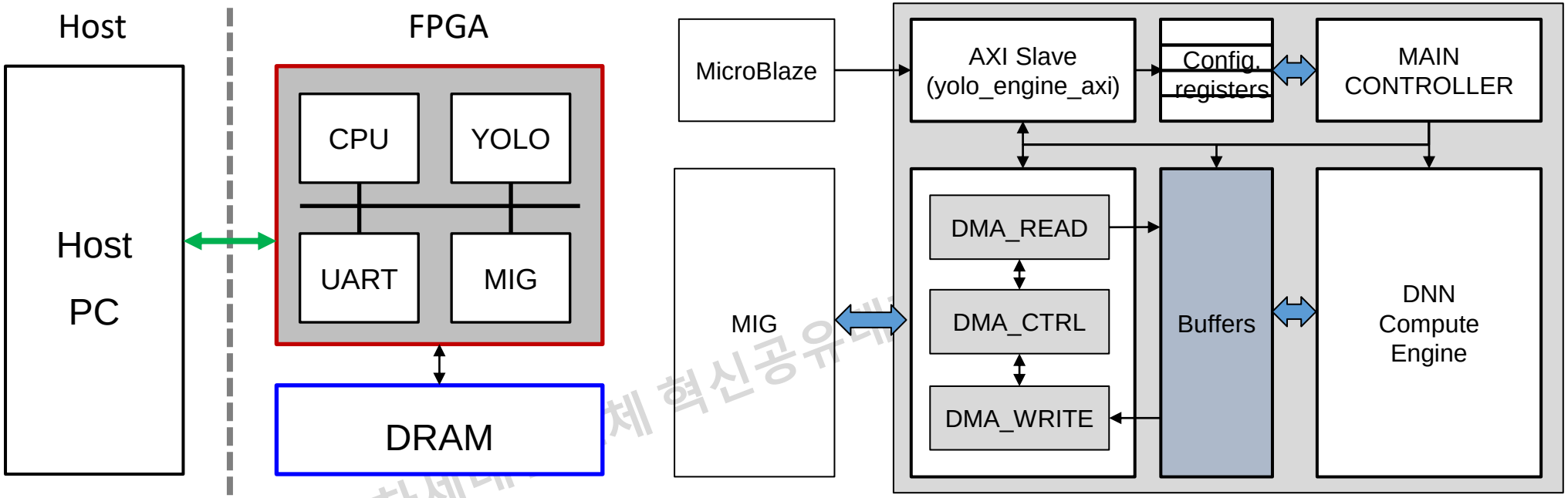
Copyright 2022 ... 체 혁신공유대학 ... reserved.

Outlines

- Recap
- System Overview
- Test Vector Generation
- Sample Top Design

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

Sample Top Module (**yolo_engine_ip**)



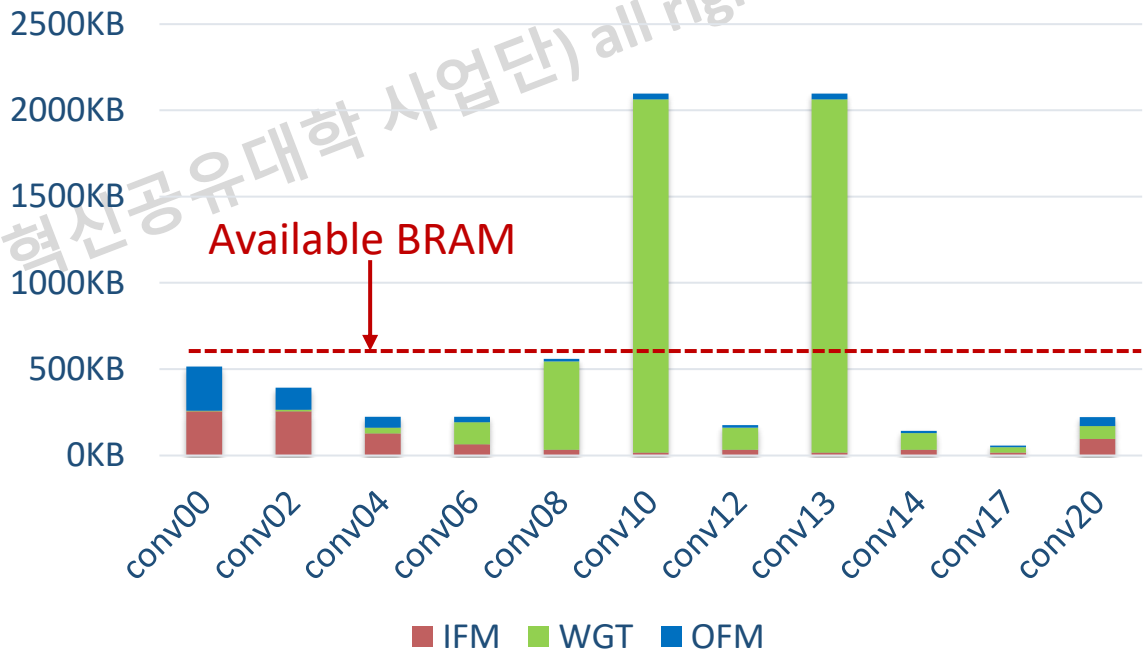
- AXI Slave: To communicate with the MicroBlaze CPU via control & status registers
- AXI Master: To transfer data between DRAM and Buffers via an AXI DMA Master
- Main controller and DNN Compute Engine
- Buffers: Weight/bias/scale buffers and feature map buffers

Copyright 2022-2026 (차세대반도체 혁신공유대학

Example Design: Weight Buffer

- Observation: Cannot store all weights in on-chip SRAM.
- Choose the size of weight buffers:
  - Number of words? The word size? Size of read and write ports?
  - Initialization? Mapping? Alignment?

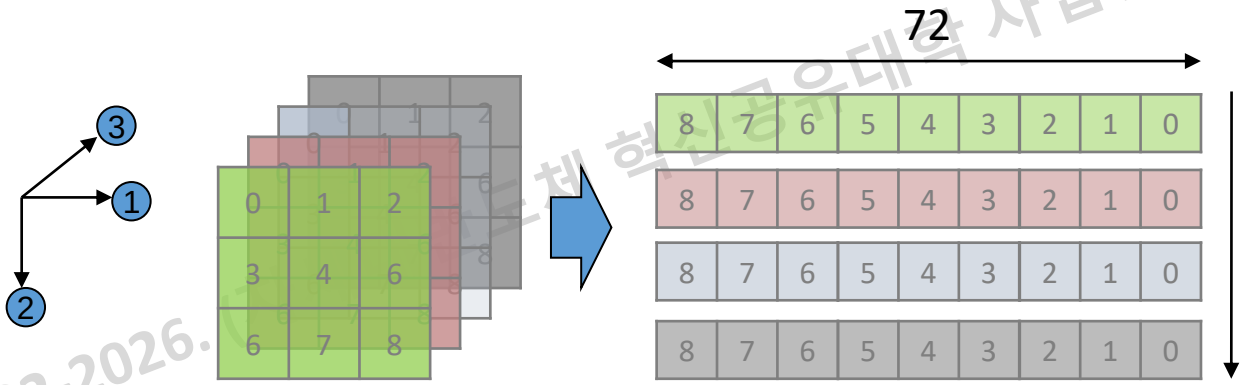
Layer	Input	Output	**Weight**
0	256×256×3	256×256×16	**3×3×3×16**
2	128×128×16	128×128×32	**3×3×16×32**
4	64×64×32	64×64×64	**3×3×32×64**
6	32×32×64	32×32×128	**3×3×64×128**
8	16×16×128	16×16×256	**3×3×128×256**
10	8×8×256	8×8×512	**3×3×256×512**
12	8×8×512	8×8×256	**1×1×512×256**
13	8×8×256	8×8×512	**3×3×256×512**
14	8×8×512	8×8×195	**1×1×512×195**
17	8×8×256	8×8×128	**1×1×256×128**
20	16×16×384	16×16×195	**1×1×384×195**



Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

Example Design: Weight Buffer

- Word size: 72 bits
  - Conv3×3: A filter 3×3×1×8b
  - Conv1×1: A filter 1×1×8×8b
- A number of words: 4096



Copyright 2022-2026. (주) 차세대반도체 혁신공유대학 사업단) all rights reserved.

Example Design: Weight Buffer

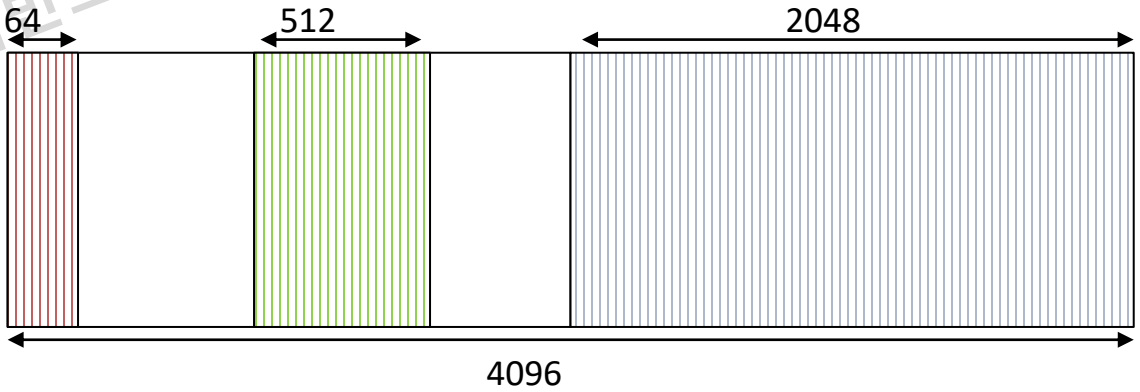
- Word size: 72 bits
  - Conv3×3: A filter 3×3×1×8b
  - Conv1×1: A filter 1×1×8×8b
- A number of words: 4096
  - Buffer size: 4096×72 = 8×(512×72) = eight 36Kb BRAMs

Layer	Input	Output	Weight	start_addr	Range	Remark
0	256×256×3	256×256×16	3×3×3×16	0	64 (=4*16)	ROM like
2	128×128×16	128×128×32	3×3×16×32	512	512 (=16*32)	ROM like
4	64×64×32	64×64×64	3×3×32×64	2048	2048 (=32*64)	ROM like

Example: Coefficient (COE) file

```
memory_initialization_radix=16;
memory_initialization_vector=
ebfe17d6ef3dfaf513,
fdfc0be0f230fbf411,
fbff06e1ef3100f010,
000000000000000000,
ef0206ee0505eff10c,
eae9e3f2fed4f5f0ed,
27240b2c3efa1a1311,
000000000000000000,
09ea051501f80e0603,
1403140e0610110908,
efd707e5d8f7e6db00,
000000000000000000,
ff09fc18170b0c1118,
fcfcf400f8fcf8fd14,
```

- Initialization
  - Layer 0: 0~63
  - Layer 1: 512~1023
  - Layer 2: 2048~4097



Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단)

Example Design: Test bench

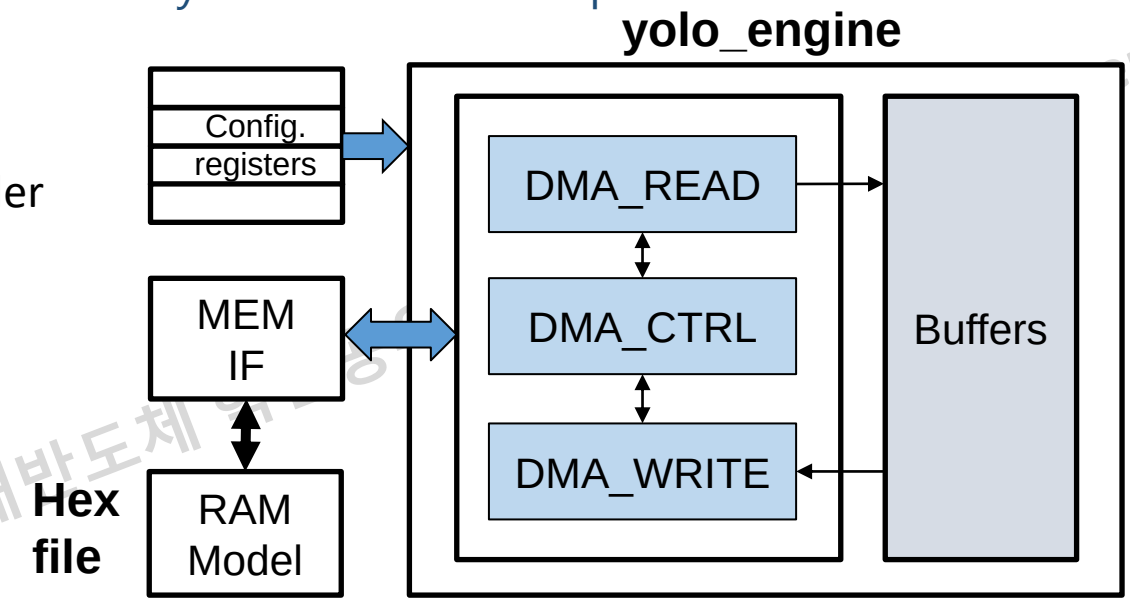
- This tests a simplified engine to **simulate a top module with an external memory model**
  - The engine has a DMA master and a DMA controller
  - The engine is connected to a memory interface via AXI ports

8 bits

1	64
2	64
3	62
4	5d
5	5c
6	58
7	58
8	56

16 bits + reorder

1	7064
2	0070
3	7064
4	0071
5	6e62
6	006e
7	6a5d
8	0067



  - The RAM model: Memory cells are initialized from hex files
    - The model supports 16 bits ONLY → **Must pack your data in 16 bits**
    - Decide a data order by yourself, for example, packing input in channel wise
      - Example: a 32-bit {00,70,70,64} is split into two 16-bit lines of {7064} and {0070}

Copyright 2023-2026 차세대반도체 혁신공유대학 All rights reserved.

Summary

- After this lecture, you can understand:
  - The settings for SDK
  - How we evaluate the design, including accuracy, performance, and power
  - How the system overview looks like
  - How to generate the test vectors from a quantized model and activations during an inference
  - An example design: weight buffer and test bench
- Based on the sample code and design, make your own design choice.

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.

Q&A

Copyright 2022-2026. (차세대반도체 혁신공유대학 사업단) all rights reserved.